MIS41110 Practical Sheet, Week 2

Covers the Week 2 lecture (formatted output, input, files, program arguments, boolean logic, functions, modules, scope) · **Released** with the Week 2 slides · **Worked** Monday of Week 3 · **Solutions** that afternoon

Solutions from your tutors after the day's last session. No `if` and no loops yet: everything here runs straight top to bottom.

Exercise 1: arguments math

Write `pair_math.py`: it reads **two decimal values as program arguments** and prints (for `python3 pair_math.py 13 5`):

```
13.0 + 5.0 = 18.0
13.0 - 5.0 = 8.0
13.0 * 5.0 = 65.0
13.0 / 5.0 = 2.6
13.0 ** 5.0 = 371293.0
Big pair (both over 10)? False
```

(a) Run it with different arguments and check the output stays correct.

(b) Use f-strings this time; week 1's `str()` gluing is retired.

(c) In a comment: what is `sys.argv[0]`, and why does your program read `[1]` and `[2]`?

(d) The last line answers *"is this a big pair?"*: both values over 10, as one printed boolean (`and`). You can **compute** the answer this week; you can't **act** on it until `if` arrives on Friday.

Exercise 2: sign the crew in

Write `crew_add.py`: each run asks for a crew member's name and role with `input()`, and **appends** one line to a file whose name arrives as a program argument. Example:

```
$ python3 crew_add.py crew.txt
Name: Vera Lindqvist
Role: glaciologist
$ python3 crew_add.py crew.txt
Name: Tom Ekwueme
Role: cook
```

Exercise 3 ☆: read the roster back (if the session runs short, finish at home)

Write `crew_list.py`: it takes the same filename argument and prints the file's contents under a header:

```
$ python3 crew_list.py crew.txt
Crew on file 'crew.txt':
Vera Lindqvist glaciologist
Tom Ekwueme cook
```

One dial you have met once: `end=` (week 1) decides what `print` adds after each value; the file's lines already carry their own newlines.

(a) Which `open()` mode did each of the two programs need, and why?

Exercise 4: the supply table

Write `supply_table.py`: store three supplies and their weights in six variables, then print an **aligned** table using format specs (`:<12`, `:>9.2f`, with one literal space between the two fields):

```
ITEM         WEIGHT KG
fuel drum       180.00
ration box       22.50
med kit           9.75
```

Exercise 5: the station math module

Write a module `station_math.py` containing four functions:

- `c_to_f(celsius)`: returns the Fahrenheit value;
- `wind_chill(temp_c, wind_kmh)`: returns the standard formula, written the way the meteorologists write it (`t` is your `temp_c`, `v` is your `wind_kmh`)
  `13.12 + 0.6215*t - 11.37*v**0.16 + 0.3965*t*v**0.16`;
- `split_shift(hours)`: returns **two values**: full watches (`hours // 4`) and leftover hours (`hours % 4`);
- `is_freezing(celsius)`: returns a **boolean**, `True` below zero (one comparison, no `if` needed).

Then write `tonight.py`, which **imports your module** and prints (no arguments, values hard-coded: `-17.1` degrees, `35.0` km/h wind, `11` duty hours):

```
Outside: -17.1 C = 1.22 F
Freezing: True
Feels like: -29.6 C in the wind
Watches: 2 full, 3 hours spare
```

(a) Why does `tonight.py` stay correct if you improve a formula inside `station_math.py`?

(b) Scope check, in a comment: can `tonight.py` read a variable created *inside* `wind_chill`? Why not?

Meaningful names, a comment where a line needs one, and run after every change.